

# INFORME DE PRUEBA

Sistema de detección de personas fuera de horario en residencial mediante YOLOv10

## 1. Objetivo de la prueba

Verificar el funcionamiento de un sistema de detección de personas basado en YOLOv10 que identifique a residentes circulando fuera del horario de descanso establecido (toque de queda), y que envíe automáticamente una alerta con la captura de la imagen a un chat de Telegram mediante un bot.

## 2. Configuración utilizada

- Modelo: YOLOv10 nano (yolov10n.pt), entrenado sobre el dataset COCO, filtrando únicamente la clase "person" (clase 0).
- Hardware: laptop con GPU NVIDIA GeForce RTX 4050 (6GB), usando PyTorch con soporte CUDA para la inferencia.
- Fuente de video: cámara integrada de la laptop, capturada con OpenCV.
- Horario de toque de queda configurado: de 10:00 p.m. a 4:00 a.m. del día siguiente.
- Confianza mínima de detección: 0.5 (50%).
- Canal de notificación: bot de Telegram (API gratuita), mediante el método sendPhoto.
- Control de envío (cooldown): 30 segundos entre alertas, para evitar el envío repetido de la misma detección.

## 3. Procedimiento de prueba

Para validar el sistema se realizaron dos escenarios de prueba: uno dentro del horario de toque de queda (donde el sistema debe generar la alerta) y otro en horario libre (donde el sistema debe detectar a la persona pero no debe enviar ninguna alerta). El código distingue ambos casos dibujando un recuadro de color distinto alrededor de la persona detectada: verde para horario libre y rojo para horario de toque de queda.

## 4. Comandos utilizados

A continuación se detallan, en orden, todos los comandos ejecutados en el Anaconda Prompt para preparar el entorno y correr el sistema.

### 4.1 Verificación de drivers NVIDIA

```
nvidia-smi
```

### 4.2 Aceptar términos de servicio de Anaconda (si lo solicita)

```
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main  
conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
```

```
conda tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/msys2
```

```
(base) C:\Users\patri>conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
accepted Terms of Service for https://repo.anaconda.com/pkgs/main
```

```
(base) C:\Users\patri>conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
accepted Terms of Service for https://repo.anaconda.com/pkgs/r
```

```
(base) C:\Users\patri>conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/msys2
accepted Terms of Service for https://repo.anaconda.com/pkgs/msys2
```

## 4.3 Creación y activación del entorno virtual

```
conda create -n yolo python=3.10 -y
conda activate yolo
```

```
(base) C:\Users\patri>conda create -n yolo python=3.10 -y
3 channel Terms of Service accepted
Retrieving notices: done
Channels:
- defaults
Platform: win-64
Collecting package metadata (repodata.json): done
Solving environment: done
```

```
Channel "defaults" has the following notices:
[info] -- Tue Jun  9 00:00:00 2026
PyTorch 2.12 with CUDA support is now available to install with your current channel (Anaconda Main). Learn more: https://pytorch.org/
```

```
(base) C:\Users\patri>conda activate yolo
```

```
(yolo) C:\Users\patri>pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
```

## 4.4 Instalación de PyTorch con soporte GPU (CUDA)

```
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu121
```

```
(yolo) C:\Users\patri>pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
Looking in indexes: https://download.pytorch.org/whl/cu121
Collecting torch
  Downloading torch-2.5.1%2Bcu121-cp310-cp310-win_amd64.whl (2449.4 MB)
    2.4/2.4 GB 14.5 MB/s 0:01:30
Collecting torchvision
  Downloading torchvision-0.20.1%2Bcu121-cp310-cp310-win_amd64.whl (6.1 MB)
    6.1/6.1 MB 1.2 MB/s 0:00:05
Collecting torchaudio
  Downloading torchaudio-2.5.1%2Bcu121-cp310-cp310-win_amd64.whl (4.1 MB)
    4.1/4.1 MB 5.0 MB/s 0:00:00
Collecting filelock (from torch)
  Downloading filelock-3.29.0-py3-none-any.whl.metadata (2.0 kB)
Collecting typing-extensions>=4.8.0 (from torch)
  Downloading typing_extensions-4.15.0-py3-none-any.whl.metadata (2.3 kB)
```

## 4.5 Verificación de que PyTorch detecta la GPU

```
python
import torch
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
exit()
```

```
(yolo) C:\Users\patri>python
Python 3.10.20 | packaged by Anaconda, Inc. | (main, Jun 11 2026, 15:13:20) [MSC v.1942 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>>
>>>
>>> print(torch.cuda.is_available())
True
>>> print(torch.cuda.get_device_name(0))
NVIDIA GeForce RTX 4050 Laptop GPU
>>> exit()
```

Resultado obtenido: True / NVIDIA GeForce RTX 4050 Laptop GPU

## 4.6 Instalación de las librerías del proyecto

```
pip install ultralytics opencv-python requests
```

```
(yolo) C:\Users\patri>pip install ultralytics opencv-python requests
Collecting ultralytics
  Downloading ultralytics-8.4.70-py3-none-any.whl.metadata (41 kB)
Collecting opencv-python
  Downloading opencv_python-4.13.0.92-cp37-abi3-win_amd64.whl.metadata (20 kB)
Collecting requests
  Using cached requests-2.34.2-py3-none-any.whl.metadata (4.8 kB)
Requirement already satisfied: numpy>=1.23.0 in c:\miniconda3\envs\yolo\lib\site-packages (from ultralytics) (2.2.6)
```

## 4.7 Ubicarse en la carpeta del proyecto

```
cd Desktop\alarma_residencial
```

## 4.8 Ejecución del programa principal

```
python detecccion.py
```

```
(yolo) C:\Users\patri\Documents\UPeU\Semestre 8\Inteligencia Artificial\alarma_residencial>python detecccion.py
Creating new Ultralytics Settings v0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\patri\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Cargando modelo YOLOv10...
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov10n.pt to 'yolov10n.pt': 16% —————Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov10n.pt to 'yolov10n.pt': 71% —————Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov10n.pt to 'yolov10n.pt': 74% —————Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov10n.pt to 'yolov10n.pt': 93% —————Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov10n.pt to 'yolov10n.pt': 100% ————— 5.6MB 11.9MB/s 0.5s
Modelo cargado correctamente.
Cámara iniciada. Presiona 'q' en la ventana de video para salir.
Error al enviar a Telegram: 400 - {'ok': False, 'error_code': 400, 'description': 'Bad Request: chat not found'}

(yolo) C:\Users\patri\Documents\UPeU\Semestre 8\Inteligencia Artificial\alarma_residencial>python detecccion.py
Cargando modelo YOLOv10...
Modelo cargado correctamente.
Cámara iniciada. Presiona 'q' en la ventana de video para salir.
Alerta enviada a Telegram correctamente.
Alerta enviada a Telegram correctamente.
Alerta enviada a Telegram correctamente.
```

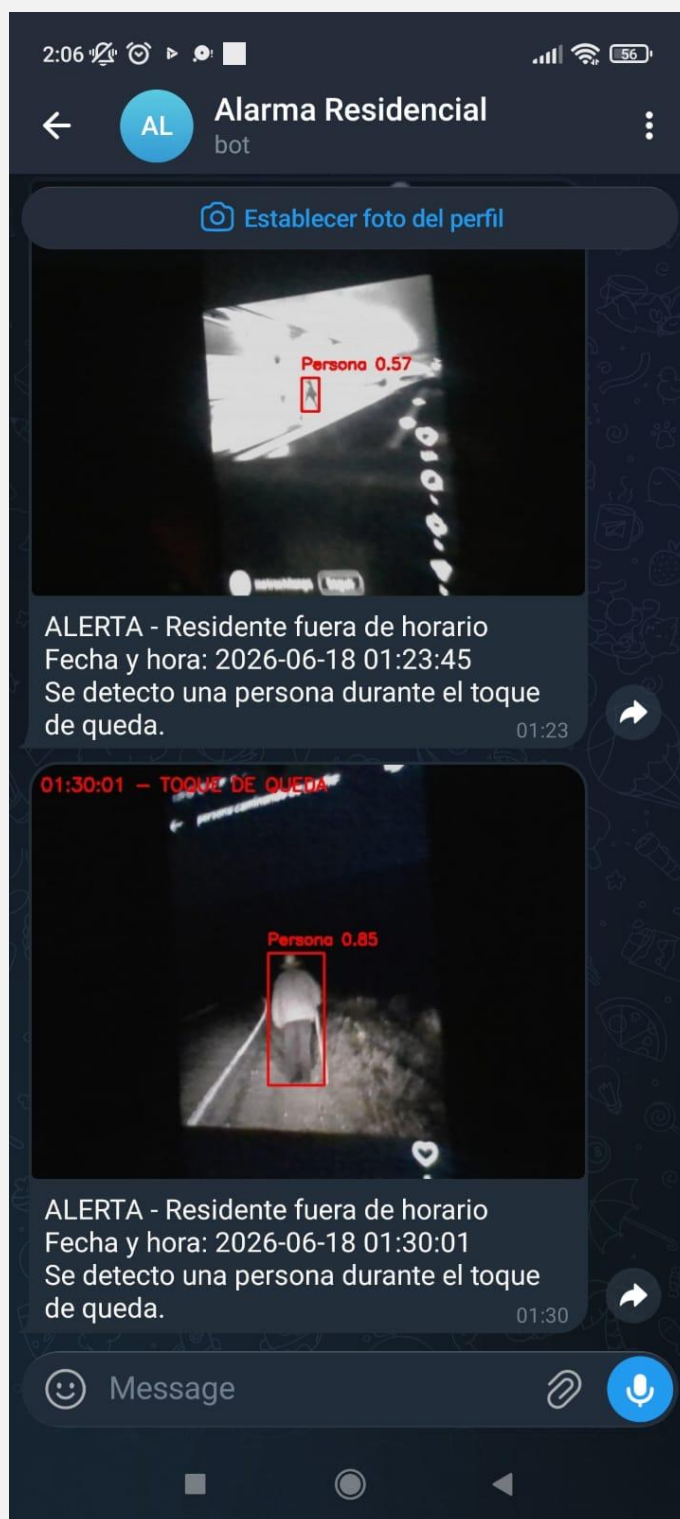
Al ejecutar este comando, el sistema descarga automáticamente el modelo yolov10n.pt (solo la primera vez), abre la cámara, y comienza a analizar cada frame en busca de personas, validando el horario y enviando la alerta a Telegram cuando corresponde.

## 5. Resultados - Escenario 1: Horario libre (fuera del toque de queda)

En este escenario el sistema detectó correctamente a la persona caminando, dibujando el recuadro verde con el porcentaje de confianza de la detección. Como se encontraba fuera del horario de toque de queda, el sistema no envió ninguna alerta a Telegram, tal como se esperaba.

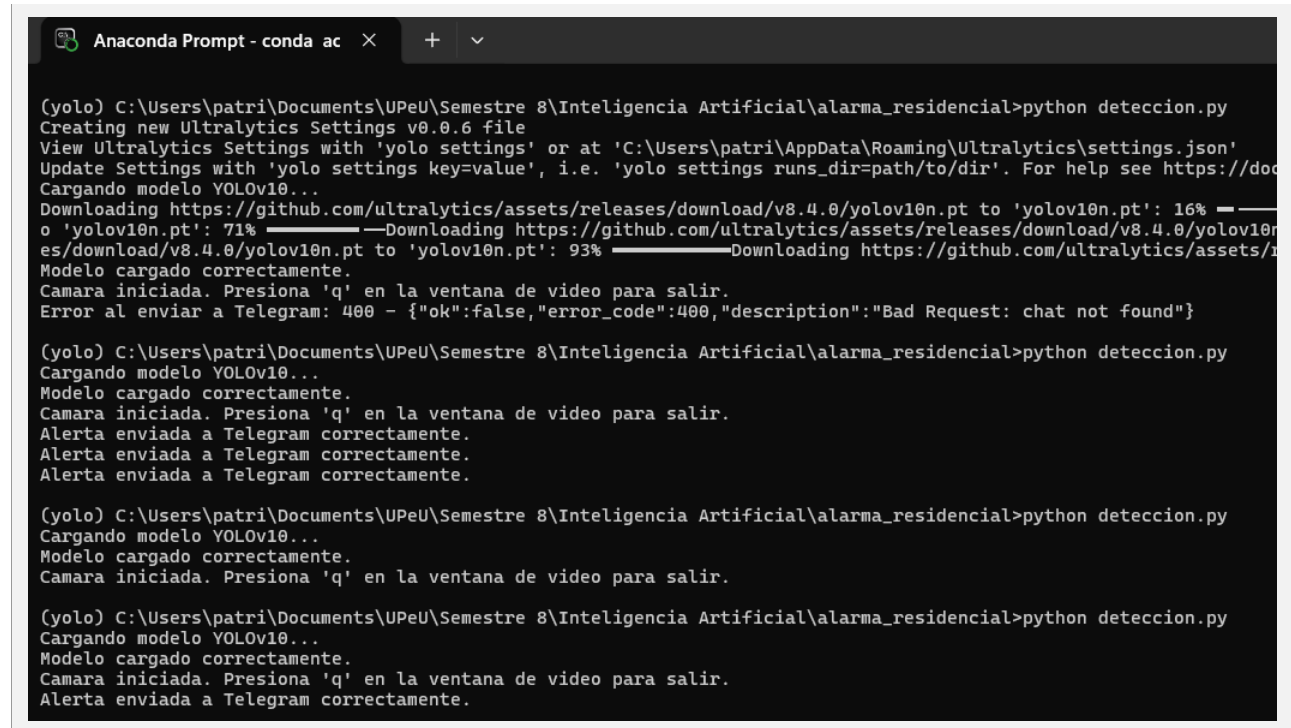


Mensaje recibido en Telegram con la foto de la alerta y el texto automático (fecha, hora y descripción).



## 7. Evidencia de ejecución en consola

A continuación se muestra la consola (Anaconda Prompt) donde se ejecutó el script, evidenciando la carga del modelo YOLOv10, el inicio de la cámara y la confirmación del envío exitoso de la alerta a Telegram.



```
Anaconda Prompt - conda ac X + v
(yolo) C:\Users\patri\Documents\UPeU\Semestre 8\Inteligencia Artificial\alarma_residencial>python deteccion.py
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at 'C:\Users\patri\AppData\Roaming\Ultralytics\settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see https://docs.ultralytics.com
Cargando modelo YOLOv10...
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov10n.pt to 'yolov10n.pt': 16%
o 'yolov10n.pt': 71% —————Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov10n
es/download/v8.4.0/yolov10n.pt to 'yolov10n.pt': 93% —————Downloading https://github.com/ultralytics/assets/1
Modelo cargado correctamente.
Camara iniciada. Presiona 'q' en la ventana de video para salir.
Error al enviar a Telegram: 400 - {"ok":false,"error_code":400,"description":"Bad Request: chat not found"}

(yolo) C:\Users\patri\Documents\UPeU\Semestre 8\Inteligencia Artificial\alarma_residencial>python deteccion.py
Cargando modelo YOLOv10...
Modelo cargado correctamente.
Camara iniciada. Presiona 'q' en la ventana de video para salir.
Alerta enviada a Telegram correctamente.
Alerta enviada a Telegram correctamente.
Alerta enviada a Telegram correctamente.

(yolo) C:\Users\patri\Documents\UPeU\Semestre 8\Inteligencia Artificial\alarma_residencial>python deteccion.py
Cargando modelo YOLOv10...
Modelo cargado correctamente.
Camara iniciada. Presiona 'q' en la ventana de video para salir.

(yolo) C:\Users\patri\Documents\UPeU\Semestre 8\Inteligencia Artificial\alarma_residencial>python deteccion.py
Cargando modelo YOLOv10...
Modelo cargado correctamente.
Camara iniciada. Presiona 'q' en la ventana de video para salir.
Alerta enviada a Telegram correctamente.
```

## 8. Conclusiones

- El sistema detectó correctamente a las personas en ambos escenarios usando el modelo YOLOv10 acelerado por GPU.
- La lógica de horario de toque de queda funcionó correctamente, diferenciando entre horario libre y horario restringido.
- El envío de alertas mediante el bot de Telegram funcionó exitosamente una vez resuelto el requisito de iniciar conversación con el bot.
- El uso de Telegram en lugar de WhatsApp permitió implementar el sistema de notificaciones sin costo, ya que la API de Telegram para bots es gratuita.